E9 222 Signal Processing in Practice
Assignment 4 — Linear Prediction

February 23, 2026

Dwaipayan Haldar

Background

Linear Prediction (LP) models a speech sample as a weighted linear combination of its N past samples:

$$\hat{x}[n] = -\sum_{k=1}^{N} a_k\, x[n-k], \tag{1}$$

giving a *prediction error* (residual) signal

$$e[n] = x[n] - \hat{x}[n] = x[n] + \sum_{k=1}^{N} a_k\, x[n-k] = \big(A(z) \star x\big)[n], \tag{2}$$

where $A(z) = 1 + a_1 z^{-1} + \cdots + a_N z^{-N}$ is the *analysis filter*. The LP coefficients are found by minimising the frame energy of the residual, leading to the **Yule–Walker** (autocorrelation) normal equations:

$$\underbrace{\begin{bmatrix} r[0] & r[1] & \cdots & r[N-1] \\ r[1] & r[0] & \cdots & r[N-2] \\ \vdots & & \ddots & \vdots \\ r[N-1] & \cdots & r[1] & r[0] \end{bmatrix}}_{\mathbf{R}} \begin{bmatrix} a_1 \\ \vdots \\ a_N \end{bmatrix} = -\begin{bmatrix} r[1] \\ \vdots \\ r[N] \end{bmatrix}, \tag{3}$$

where $r[k] = \sum_n x[n]\, x[n-k]$ is the (biased) autocorrelation of the windowed frame. A key property guaranteed by the Levinson–Durbin recursion is *monotone decrease of prediction error energy*:

$$E_N = E_{N-1}\big(1 - k_N^2\big) \le E_{N-1}, \tag{4}$$

where k_N is the N-th reflection coefficient $|k_N| \le 1$. Equivalently, the **prediction gain**

$$G_N = 10 \log_{10}\left(\frac{E_x}{E_N}\right) \text{ dB} \tag{5}$$

is non-decreasing with order N.

The speech spectrum is modelled by the all-pole (AR) synthesis filter:

$$H(z) = \frac{\sqrt{E_N}}{A(z)} = \frac{\sqrt{E_N}}{1 + \sum_{k=1}^{N} a_k z^{-k}}, \tag{6}$$

so the estimated power spectral density (PSD) is

$$\hat{S}_{xx}(e^{j\omega}) = \frac{E_N}{|A(e^{j\omega})|^2}. \tag{7}$$

5. Implementing Linear Prediction

Setup

Each audio file is processed frame-by-frame using a **25 ms Hamming window** and a **10 ms hop**. At $f_s = 16\,\text{kHz}$ this gives frames of 400 samples and a hop of 160 samples. Two solvers are implemented:

- **Inverse method** – direct matrix inversion of the Toeplitz system in (3).
- **Levinson–Durbin recursion** – exploits Toeplitz structure for $\mathcal{O}(N^2)$ complexity and guarantees the stability of $A(z)$ through the reflection-coefficient update.

Key helper functions are listed below.

```python
def autocorr(frame, order):
    r_full = np.correlate(frame, frame, mode='full')
    mid = len(frame) - 1
    return r_full[mid:mid+order+1]

def lp_coeffs_inverse(frame, order):
    r = autocorr(frame, order)
    reg = 1e-6 * r[0]                          # Tikhonov regularisation
    R = toeplitz(r[:-1]) + reg * np.eye(order)
    a = -np.linalg.solve(R, r[1:])
    err = max(r[0] + np.dot(a, r[1:]), EPS)
    return a, float(err)

def lp_coeffs_levinson(frame, order):
    r = autocorr(frame, order)
    a = np.zeros(order + 1);  a[0] = 1.0
    e = float(r[0])
    for i in range(1, order + 1):
        acc = r[i] + np.dot(a[1:i], r[i-1:0:-1])
        k   = -acc / max(e, EPS)               # reflection coefficient
        a_prev = a.copy()
        for j in range(1, i):
            a[j] = a_prev[j] + k * a_prev[i-j]
        a[i] = k
        e = max(e * (1 - k*k), EPS)            # Eq. (4): monotone decrease
    return a[1:], float(e)
```

Listing 1: Autocorrelation and LP coefficient solvers

```python
def residual_signal(frame, a):
    order = len(a)
    e = np.zeros_like(frame)
    for n in range(len(frame)):
```

```
        v = frame[n]
        for i in range(1, order + 1):
            if n - i >= 0:
                v += a[i-1] * frame[n-i]
        e[n] = v
    return e
```

Listing 2: Residual computation

Question 5(a) – Residual Energy per STFT Frame

For each STFT frame the Hamming-windowed segment is passed through the analysis filter $A(z)$ to obtain the residual; its energy is $E_N^{(m)} = \|e^{(m)}\|^2$. The **prediction gain** (Eq. (5)) measures how well LP whitens the frame spectrum.

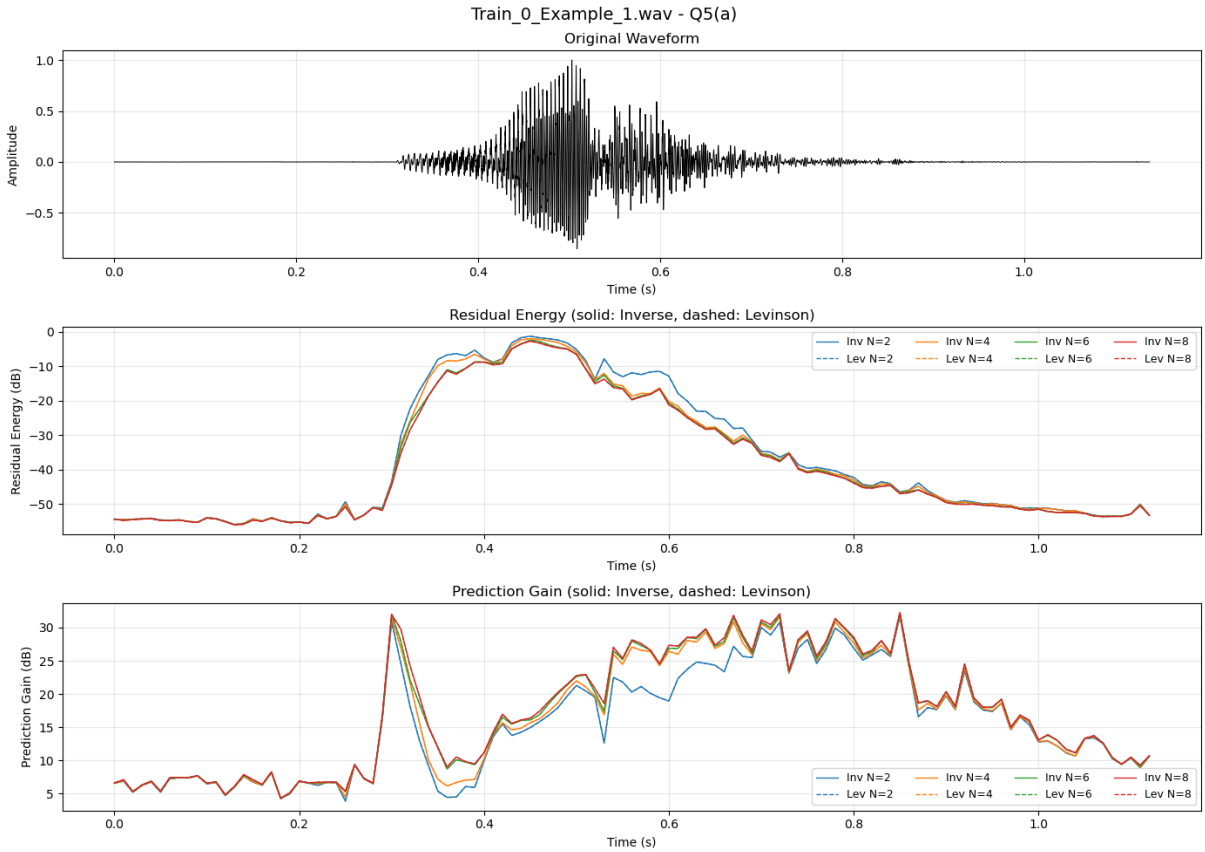


Figure 1: Q5(a) – `Train_0_Example_1.wav`. *Top*: original waveform. *Middle*: residual energy (dB) per frame. *Bottom*: prediction gain (dB) per frame. Solid lines: Inverse method; dashed: Levinson–Durbin.

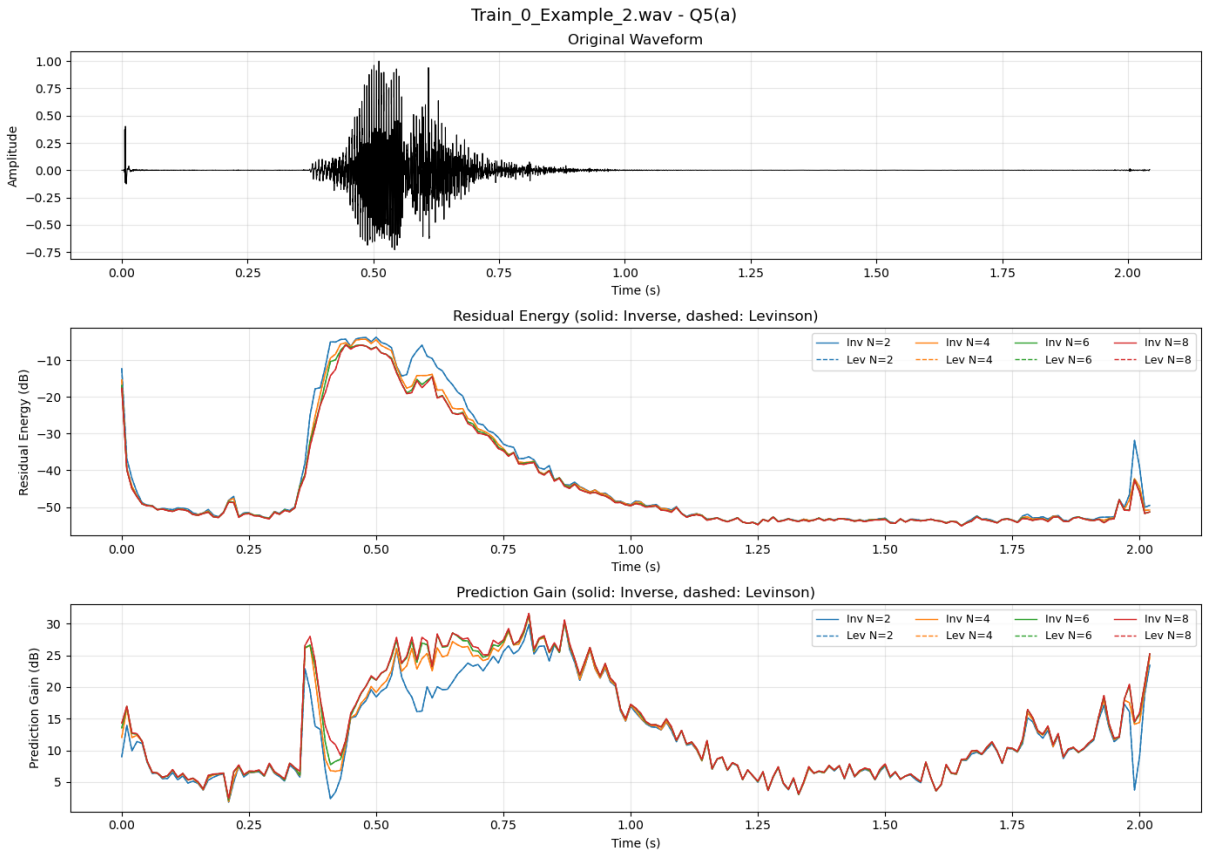


Figure 2: Q5(a) – `Train_0_Example_2.wav`.

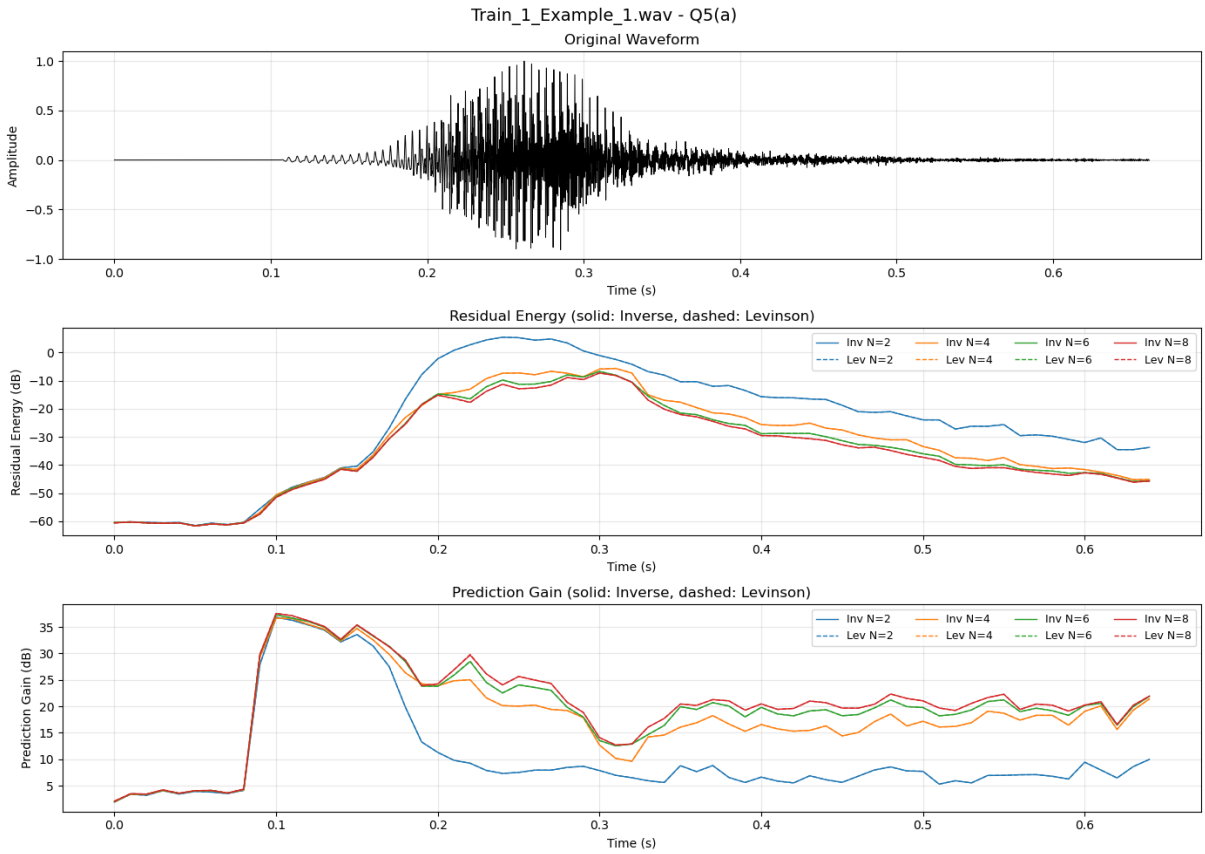


Figure 3: Q5(a) – `Train_1_Example_1.wav`.

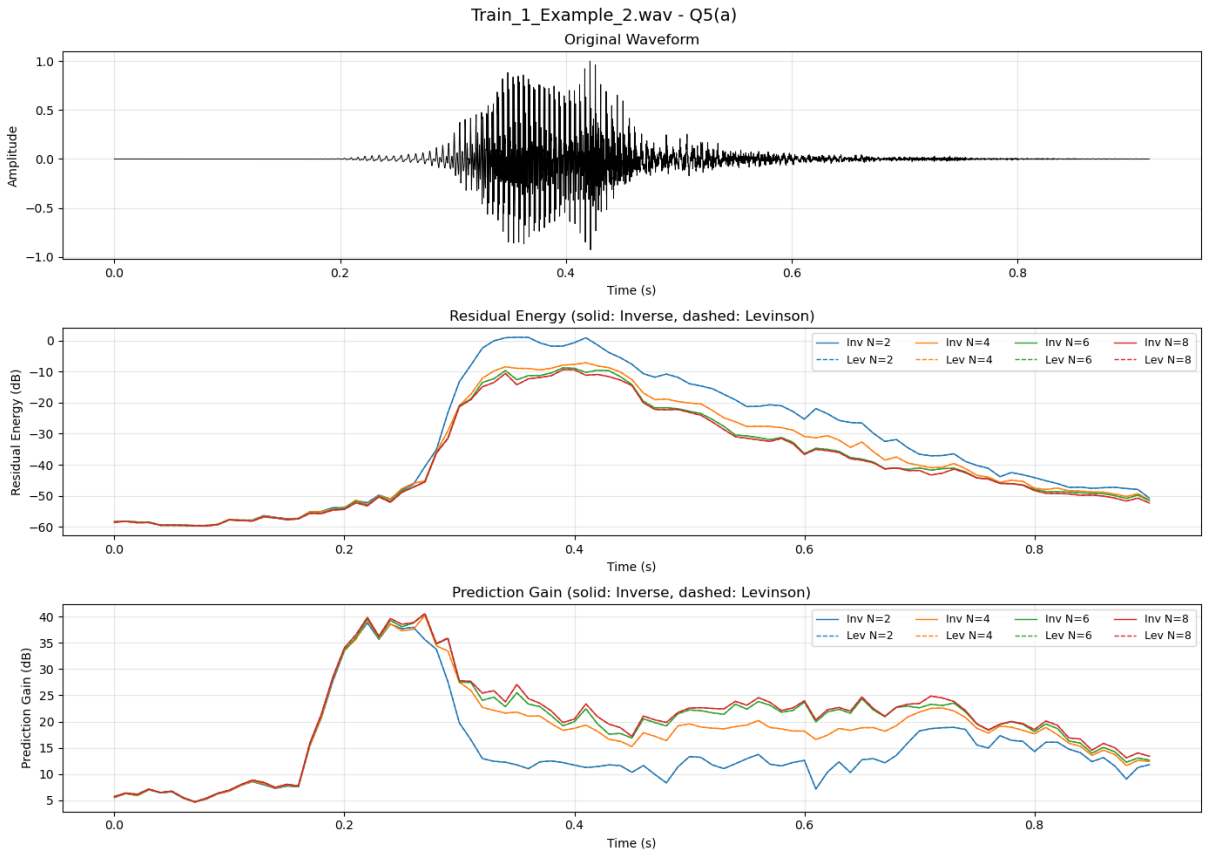


Figure 4: Q5(a) – `Train_1_Example_2.wav`.

Observations – Q5(a)

1. **Residual energy tracks speech activity.** Frames with silence (before or long after the main utterance) have very low residual energy ($\lesssim -45\,\mathrm{dB}$), whereas actively voiced frames have residual energies close to -5 to $0\,\mathrm{dB}$. This is expected: during silence there is almost nothing to predict, so the raw energy is tiny.

2. **Voiced vs. unvoiced discrimination via prediction gain.** Voiced frames exhibit *high prediction gain* (15–40 dB): the quasi-periodic, highly correlated glottal excitation makes the spectrum strongly peaked, and LP can model those formant peaks well with a small residual. Unvoiced (fricative) frames, whose spectrum is nearly flat (noise-like), yield low prediction gain ($\approx$ 5–10 dB), as LP finds little autocorrelation structure to exploit. This connection is key to classic LP-based voiced/unvoiced detectors.

3. **Higher order N lowers residual energy.** Consistent with Eq. (4), adding more predictor taps always decreases (or at worst preserves) the prediction error energy. The improvement from $N = 2 \to 4$ is large; beyond $N = 6$ the gain curves are nearly superimposed, suggesting saturation for speech at 16 kHz (roughly 2–3 formants captured by $N = 6$–8).

4. **Inverse method $\approx$ Levinson–Durbin.** The solid and dashed curves of each colour are practically indistinguishable for all four files: both solvers minimise the same quadratic cost (Eq. (3)) and differ only in numerical implementation.

Question 5(b) – Signal Reconstruction via AR Model

The AR model reconstructs each frame spectrum as:

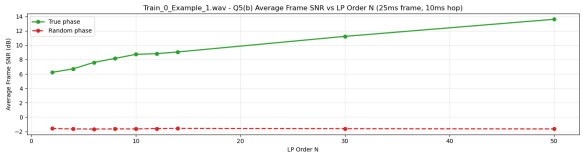
$$\hat{X}(e^{j\omega}) = \sqrt{E_N}\,\frac{1}{A(e^{j\omega})} \cdot e^{j\phi(\omega)}, \tag{8}$$

where $\phi(\omega)$ is either the *original STFT phase* (true-phase case) or a *uniformly random phase* (random-phase case). The inverse STFT reassembles the signal with overlap-add (OLA).

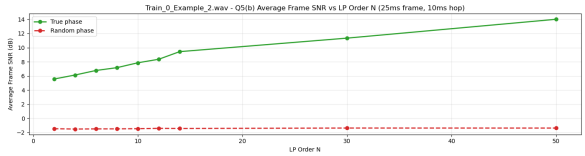
```python
def reconstruct_signal(frames, starts, frame_len, solver, order, orig_x):
    win   = np.hamming(frame_len)
    n_fft = frame_len;  n_bins = n_fft // 2 + 1
    y_orig, y_rand, win_sum = (np.zeros(starts[-1]+frame_len) for _ in range(3))
    rng = np.random.default_rng(0)

    for i, frame in enumerate(frames):
        s  = starts[i]
        xw = frame * win
        X  = np.fft.rfft(xw, n=n_fft)
        phase      = np.angle(X)
        rand_phase = rng.uniform(-np.pi, np.pi, size=n_bins)

        a, err_var = solver(xw, order)
        den = np.concatenate(([1.0], a))
        _, H = freqz([1.0], den, worN=n_bins)
        mag_env = np.sqrt(max(err_var, EPS)) * np.abs(H)  # AR envelope

        Xo = mag_env * np.exp(1j * phase)          # true phase
        Xr = mag_env * np.exp(1j * rand_phase)     # random phase

        xo = np.fft.irfft(Xo, n=n_fft)[:frame_len]
        xr = np.fft.irfft(Xr, n=n_fft)[:frame_len]

        y_orig[s:s+frame_len] += xo * win
        y_rand[s:s+frame_len] += xr * win
        win_sum[s:s+frame_len] += win**2            # OLA normalisation

    y_orig /= np.maximum(win_sum, EPS)
    y_rand /= np.maximum(win_sum, EPS)
    n = min(len(orig_x), len(y_orig))
    return y_orig[:n], y_rand[:n]
```

Listing 3: Signal reconstruction via overlap-add with original and random phase

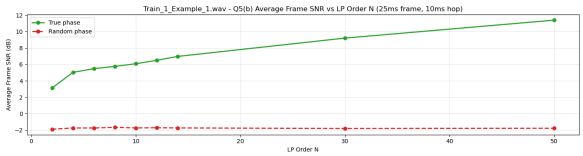
SNR vs. LP Order



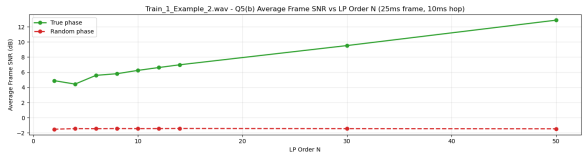
(a) `Train_0_Example_1.wav`



(b) `Train_0_Example_2.wav`



(c) `Train_1_Example_1.wav`



(d) `Train_1_Example_2.wav`

Figure 5: Q5(b) – Average frame SNR of AR-reconstructed signal vs. LP order N for true-phase (green) and random-phase (red) excitation. LP orders tested: $N \in \{2, 4, 6, 8, 10, 12, 14, 30, 50\}$.

Waveform and Spectrogram Comparison ($N = 50$)

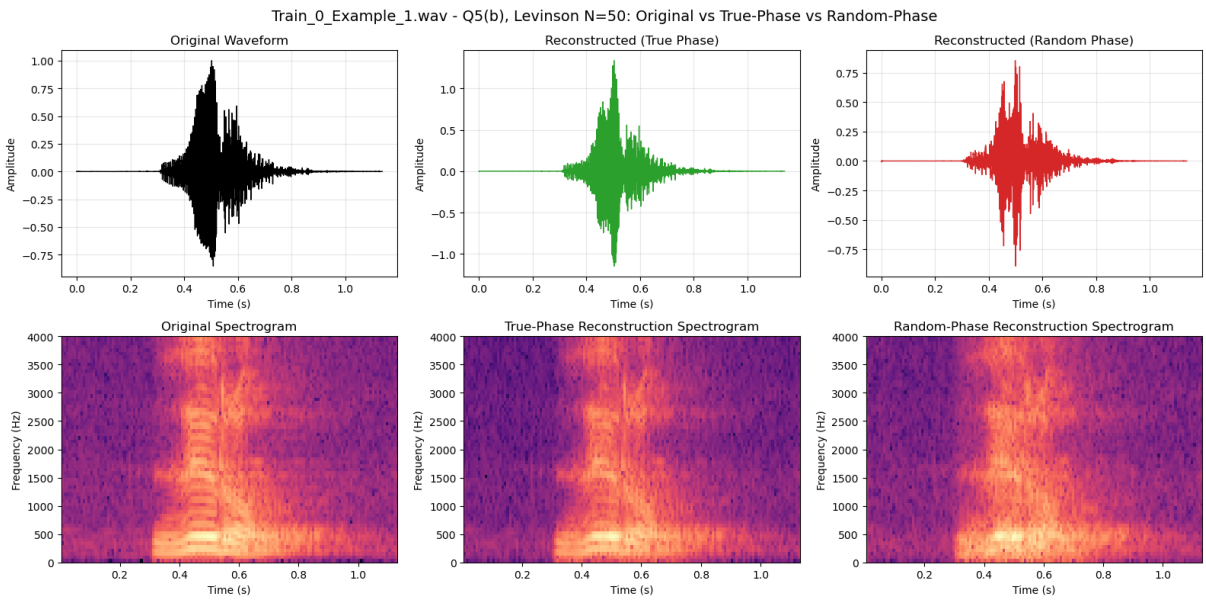


Figure 6: Q5(b) – `Train_0_Example_1.wav`, Levinson $N = 50$. *Top row*: original, true-phase, and random-phase reconstructed waveforms. *Bottom row*: corresponding spectrograms.

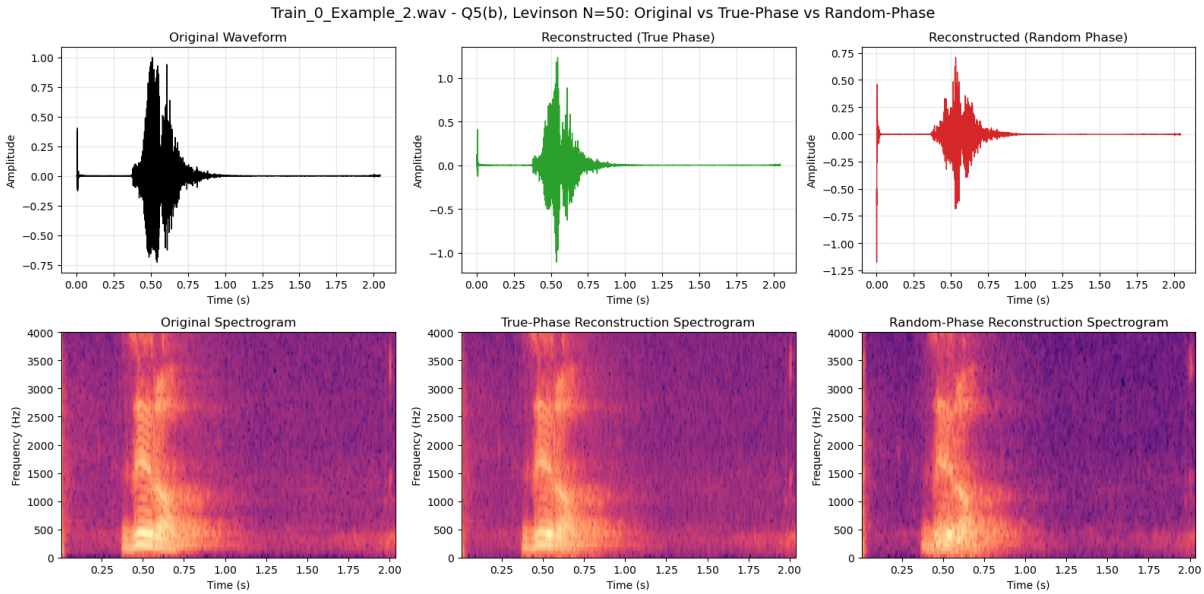


Figure 7: Q5(b) – `Train_0_Example_2.wav`, Levinson $N = 50$.

Observations – Q5(b)

1. **True-phase reconstruction improves with N.** The average frame SNR for the true-phase case increases monotonically from $\approx 6\,\text{dB}$ at $N = 2$ to $\approx 13\,\text{dB}$ at $N = 50$ across all four files (Fig. 5). The slope is largest for small N (where adding one more pole adds a new formant) and flattens beyond $N \approx 15$, suggesting that eight or so poles capture most of the spectral envelope information. No strict saturation is observed up to $N = 50$.

2. **Random-phase reconstruction is consistently $\approx -1.5\,\text{dB}$ SNR.** Scrambling the spectral phase destroys temporal fine structure (the glottal pulses), resulting in a noisy, breathy output whose SNR barely changes with N. This shows that for a *waveform* fidelity metric the phase is at least as important as the spectral envelope.

3. **Spectrograms reveal what SNR does not.** Even though the random-phase reconstructed signal has very low SNR, its spectrogram (Figs. 6–7) preserves the formant structure (horizontal bands) quite well. This reflects the fact that LP correctly models the spectral *envelope*; it is only the excitation (voiced periodicity) that is lost.

4. **True-phase reconstruction preserves pitch structure.** The true-phase waveform shows clear glottal pulse patterns in voiced regions (visible as periodic spikes), while the random-phase version produces a smoothed, diffuse envelope – similar to whispering or spectrally shaped noise. This dichotomy is the basis for LP vocoders.

5. **Quality does not *saturate* quickly.** Unlike the residual energy (Q5a) which plateaus by $N \approx 8$, reconstruction SNR continues to grow noticeably between $N = 14$ and $N = 50$. This is because SNR measures absolute waveform differences, which are reduced by finer spectral envelope fitting even when the perceptual improvement is small.

6. Spectral Transform Linear Prediction (STLP)

Formulation

Standard LP ($q = 1$) solves the Yule–Walker system using the autocorrelation sequence of $x[n]$, which equals the inverse DTFT of $S_{xx}(e^{j\omega})$. The **Spectral Transform LP** generalises this by computing the modified autocorrelation as the inverse DTFT of $S_{xx}^{1/q}(e^{j\omega})$:

$$\tilde{r}_q[k] = \mathrm{IDTFT}\{S_{xx}(e^{j\omega})^{1/q}\} = \frac{1}{2\pi}\int_{-\pi}^{\pi} S_{xx}(e^{j\omega})^{1/q}\, e^{j\omega k}\, d\omega. \tag{9}$$

Solving the Yule–Walker system with $\tilde{r}_q[k]$ yields LP coefficients $\hat{a}^{(q)}$ and residual variance $\hat{E}_q$, giving the AR model of $S_{xx}^{1/q}$:

$$\hat{S}_{xx}^{1/q}(e^{j\omega}) = \frac{\hat{E}_q}{|A_q(e^{j\omega})|^2}. \tag{10}$$

Raising to the power q recovers the estimate of the original PSD:

$$\hat{S}_{xx}(e^{j\omega}) = \left[\hat{S}_{xx}^{1/q}(e^{j\omega})\right]^q = \left[\frac{\hat{E}_q}{|A_q(e^{j\omega})|^2}\right]^q. \tag{11}$$

For $q = 1$ this reduces to the standard LP PSD of Eq. (7). Increasing q effectively *compresses* the spectral dynamic range before fitting, encouraging the LP model to allocate poles uniformly across frequency rather than only at the strongest formants.

```python
def transform_lp(frame, fs, order, q):
    xw  = frame * win
    xw /= np.sqrt(np.sum(win**2) + EPS)          # normalise by window energy

    X   = np.fft.fft(xw)
    Sxx = np.abs(X)**2
    Sxx = Sxx / (np.max(Sxx) + EPS)              # numerical stability

    S_q  = Sxx ** (1.0 / q)                      # power-law compression
    r_q  = np.real(np.fft.ifft(S_q))             # modified autocorrelation

    a, err = yw_from_acf(r_q, order)             # Yule-Walker solve

    A       = np.fft.fft(np.concatenate(([1.], a)), n=len(Sxx))
    S_q_hat = err / (np.abs(A)**2 + EPS)         # AR model of S^{1/q}

    S_hat   = np.maximum(S_q_hat, EPS) ** q      # raise to power q

    f    = np.fft.rfftfreq(len(Sxx), 1/fs)
    half = len(f)
```

```
    return f, Sxx[:half], S_hat[:half]
```

Listing 4: STLP implementation (`transform_lp`)

Frame Selection

A voiced and an unvoiced frame are selected from `Train_0_Example_1.wav` using the Levinson residual energy profile (order $N = 8$) as a proxy: the frame with *minimum* residual energy is deemed voiced (strong glottal correlation allows accurate prediction), and the frame with *maximum* residual energy is unvoiced (noise-like spectrum, poor predictability).

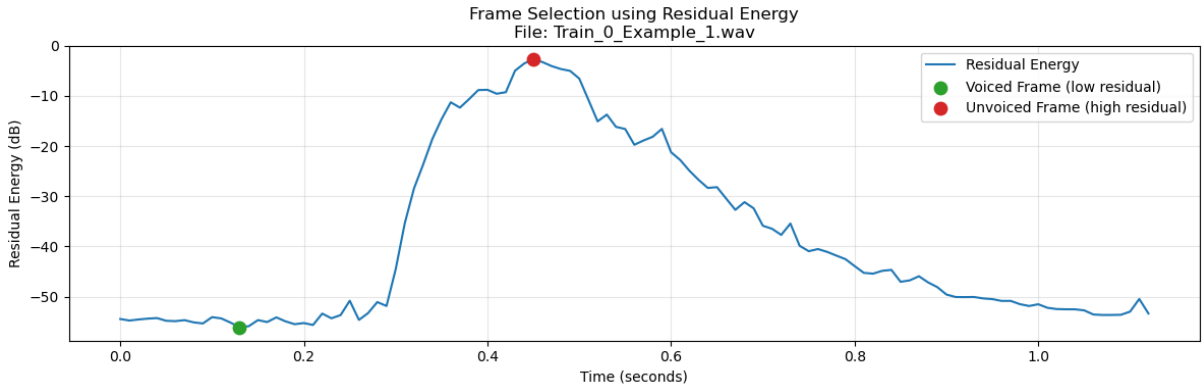


Figure 8: Frame selection using residual energy profile. The green dot marks the selected voiced frame (near silence onset, very low residual); the red dot marks the selected unvoiced frame (peak of speech activity – highest residual).

Note on frame labelling: The green dot (minimum residual energy) falls in the pre-speech silence region. During silence the signal is near zero, so the LP residual is also nearly zero – this is why the algorithm picks it. In practice, voiced/unvoiced detection should be applied only within speech activity regions; here it serves to illustrate the two extremes of residual energy.

PSD Estimation for $q = 1, 2, 3$

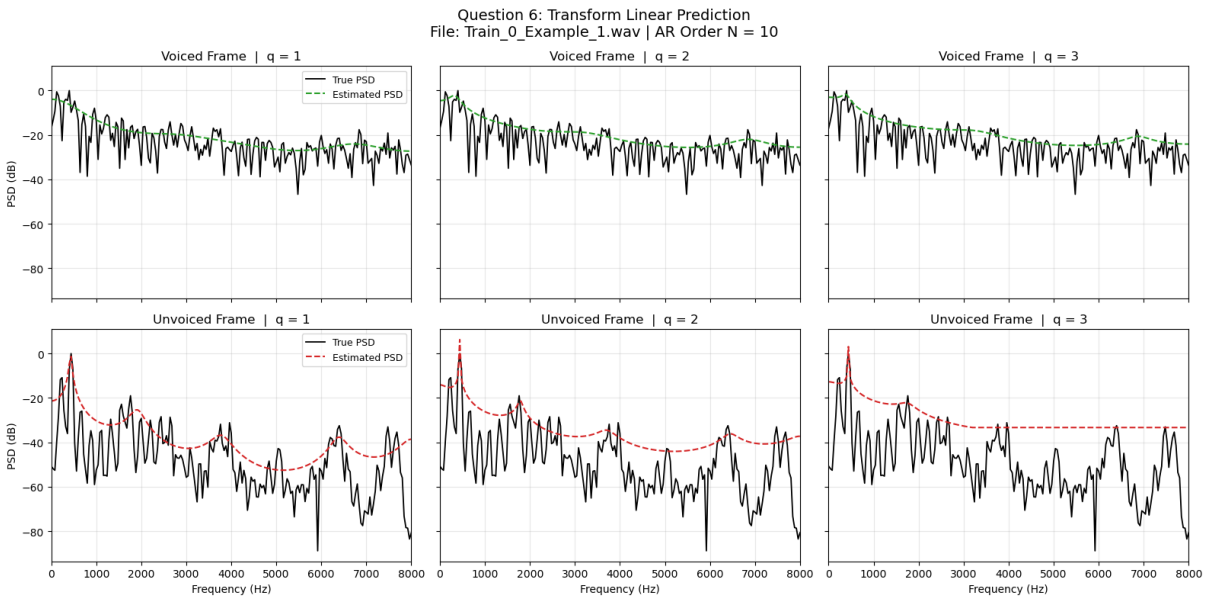


Figure 9: Q6 – Transform LP PSD estimates for $q = 1, 2, 3$ and AR order $N = 10$. *Top row*: voiced frame. *Bottom row*: unvoiced frame. Black: true PSD; green/red dashed: $\hat{S}_{xx}$ from STLP.

Observations – Q6

1. $q = 1$ **(standard LP) focuses on high-energy regions.** Because the Yule–Walker system is solved directly on the raw PSD, the AR model allocates its poles to match the dominant spectral peaks (formants in voiced frames, or broadband energy in unvoiced frames). For voiced frames the estimate captures the two or three largest formant peaks but can miss lower-energy formants or valley regions.

2. $q = 2$ **compresses the PSD dynamic range.** Taking the square root of the PSD ($S_{xx}^{1/2}$) before solving attenuates the dominance of the strongest spectral peaks. As a result, the AR model distributes its poles more evenly across the spectrum. The estimate for voiced frames shows a somewhat smoother envelope that better follows medium-energy formants, though the peak heights are less accurately captured.

3. $q = 3$ **leads to over-smoothing.** The cube-root compression ($S_{xx}^{1/3}$) strongly suppresses high-energy peaks. The resulting LP model fits the compressed spectrum well, but after the $(\cdot)^3$ inverse transform the estimated PSD tends to be nearly flat (over-smoothed). For voiced frames, formant peaks are essentially washed out; for unvoiced frames (which are already flatter), the estimate is comparable to $q = 1$.

4. **Voiced vs. unvoiced frames respond differently.** For the *voiced* frame, $q = 1$ gives the sharpest and most interpretable formant peaks. As q increases the model loses the ability to resolve fine spectral detail because the compression brings all frequency bins to similar amplitudes. For the *unvoiced* frame the spectrum is already relatively flat, so varying q has a smaller effect – all three models produce roughly similar, smooth PSD estimates.